

OMNIX RADIANCE

v0.6 Illumination Foundation

Integrated Architecture and Implementation Tasklist

Prepared for the Omnix Engine / Last Transistor rendering stack
Architecture-first, render-graph-integrated, agent-executable plan
Version 0.6 planning document - July 2026

CORE DIRECTIVE

Radiance illumination extends the existing Omnix Renderer, GPUScene, RenderGraph, GBuffer, materials, ECS and editor viewport. It never becomes a duplicate renderer with a second frame loop, scene, depth buffer or material database.

Document Control

Field	Value
Document	Omnix Radiance v0.6 - Illumination Foundation
Role	Integrated architecture and agent-executable implementation tasklist
Current baseline	Omnix Engine v0.4 deferred renderer with RenderGraph, GPUScene, GBuffer A-D, SSAO, clustered light culling, sun shadow map, HDR color and editor presentation
Geometry prerequisite	Omnix Radiance v0.5 RVG contracts and conventional fallback path
Primary output	A shared linear-HDR illumination pipeline consumed by conventional and RVG geometry
Non-goals	Full real-time GI, path tracing, neural rendering, production hair/water/subsurface systems, ray tracing as a mandatory dependency
Delivery rule	Every milestone leaves the renderer buildable, visible, diagnosable and safe under reduced capability or capacity

How to Use This Document

Treat every I-milestone as a vertical implementation slice. A milestone is complete only when its resource contracts, pass integration, editor controls, debug views, validation cases, failure behaviour and exit gates are all present.

Engineering rule

Never hide a broken surface, shadow, reflection or exposure result behind post-processing. Inspect intermediate results independently, then optimize only after parity with a simple reference path.

Contents

- 1. Document Purpose
- 2. Existing Renderer Baseline
- 3. Proposed Integrated Illumination Architecture
- 4. Global Engineering Rules
- 5. Development Milestones
- 6-21. I0-I15 Implementation Milestones
- 22. Required Radiance Debug Panel
- 23. Required Visualization Modes
- 24. Recommended Source Structure
- 25. Recommended Implementation Sequence
- 26. Premature Implementations to Avoid
- 27. Final Completion Criteria
- Appendices A-C

1. Document Purpose

This document defines the complete implementation plan for the Omnix Radiance v0.6 illumination layer. It converts the current geometry-first viewport and partially implemented deferred-lighting path into a physically coherent, inspectable, scalable and presentation-ready renderer.

- Formal shared surface representation for conventional geometry and future RVG material resolve.
- Stable metallic/roughness physically based material model.
- Directional, point and spot direct lighting with reference and clustered paths.
- Stable directional cascades and budgeted local-light shadows.
- HDR sky, diffuse irradiance and roughness-aware specular IBL.
- Correct SSAO/GTAO ambient visibility.
- Reflection probes and confidence-driven SSR.
- One-way linear-HDR exposure, bloom, tone-map and display pipeline.
- Velocity, temporal history validation and TAA.
- Analytic fog and scalable volumetric lighting.
- Editor authoring, resource inspection, GPU timing, capacity telemetry and safe fallbacks.

Integrated-renderer requirement

Radiance illumination is a new capability inside the current Omnix Renderer. It consumes existing scene extraction, GPUScene, geometry submission, RenderGraph, viewport resources, materials and editor presentation rather than duplicating them.

2. Existing Renderer Baseline

The current frame lifecycle is owned by Renderer and uses BeginFrame, RenderFrame and EndFrame. Scene components are extracted into a CPU-side render scene, visible queues are built and sorted, GPUScene is updated, passes are registered into RenderGraph, the graph executes in dependency order, and the graphics queue is submitted for presentation.

Current frame lifecycle

```

Renderer::BeginFrame()
-> wait frame fence
-> acquire swapchain image
-> reset command pool

Renderer::RenderFrame()
-> recreate viewport resources when required
-> extract ECS render scene
-> build opaque / transparent queues
-> visibility cull and sort
-> update GPUScene
-> register and compile RenderGraph
-> ExecuteWithValidation()

Renderer::EndFrame()
-> submit command buffers
-> present swapchain image
-> advance frame index

```

2.1 Active baseline passes

Pass	Current responsibility	v0.6 treatment
ShadowPass	Single directional depth map	Upgrade to multi-view cascades and local-shadow infrastructure.
DepthPrepass	Optional scene depth	Retain one shared depth and formalize its convention.
GBufferPass	Writes GBuffer A-D and depth	Formalize surface semantics; add object ID and velocity.
SSAOPass / AOBlurPass	Current screen-space AO	Audit, correct and integrate only with indirect visibility.

Pass	Current responsibility	v0.6 treatment
LightCullingPass	Clustered local-light lists	Validate against reference lighting and harden overflow.
DeferredLightingPass	Reads GBuffer and writes HDRColor	Refactor into PBR direct and environment lighting.
TransparentPass	Forward transparent rendering into HDR	Retain after opaque lighting and atmosphere ordering.
PostProcessPass	Tone mapping to LDR	Expand into exposure, bloom, tone-map and color pipeline.
EditorOverlayPass	Selection and gizmos; partly stubbed	Repair object ID and preserve overlay order.
UIPass / PresentPass	ImGui and presentation	Remain existing end-of-frame ownership.

2.2 Existing frame resources

Resource	Format	Current semantic
DepthBuffer	D32_SFLOAT	Main depth sampled by SSAO and lighting.
GBufferA	R8G8B8A8_UNORM	Base color RGB; alpha underused.
GBufferB	R16G16B16A16_SFLOAT	Normal RGB and roughness A.
GBufferC	R8G8B8A8_UNORM	Metallic R, AO G, invalid ID placeholder B.
GBufferD	R8G8B8A8_UNORM	Emissive RGB and shading model A.
ShadowMap	D32_SFLOAT	Single 2048x2048 sun shadow.
SSAO / SSAOBlurred	R8_UNORM	Raw and filtered ambient visibility.
HDRColor	R16G16B16A16_SFLOAT	Pre-tonemap scene color.
LDRColor / ViewportColor	Swapchain-compatible	Post-process and editor output.

2.3 Geometry convergence

Conventional meshes render through the indexed path. RVG assets currently use their conventional fallback mesh because GPU-driven RVG passes are not active in the frame graph. Both therefore converge in the standard GBuffer pass. Future RVG visibility reconstruction must write the same surface contract; illumination must never branch on geometry origin.

Conventional mesh -----> Shared GBuffer -> Shared Radiance
 RVG asset -> conventional fallback mesh --/

2.4 Critical baseline defects

Critical: depth reconstruction

Projection and sampled-depth conventions must be centralized before any world-position-dependent lighting, SSAO, SSR or fog result can be trusted.

Critical: object ID

The outline shader expects an unsigned object-ID texture while the descriptor is bound to a floating-point shadow image, and GBufferC writes zero where picking expects an ID. A dedicated R32_UINT attachment is mandatory.

High: render-thread ECS access

Renderer directly querying mutable ECS state can create synchronization and lifetime risks. A stable render snapshot should feed GPUScene update and pass registration.

3. Proposed Integrated Illumination Architecture

3.1 Architectural position

```
ECS Scene
-> RenderSceneExtractor
-> Visibility / Queue Construction
-> GPUScene::UpdateFrame
-> Renderer::setupRenderGraph
  -> Geometry and Surface Generation
  -> Shadow Visibility
  -> Clustered Light Preparation
  -> Radiance Illumination
  -> Atmospheric Transport
  -> Camera and Temporal Pipeline
  -> Existing Editor Overlay
  -> Existing UI and Presentation
```

Radiance begins after visible geometry has produced trustworthy surface information and ends before editor overlays, UI and presentation. It is a group of passes and registries integrated through the existing RenderGraph, not a second top-level renderer.

3.2 Proposed complete frame

```
1. Scene Extraction and GPUScene Synchronization
2. Geometry and Surface Generation
  -> Depth Prepass -> GBuffer + Object ID + Velocity + Scene Depth
3. Parallel Lighting Preparation
  -> Directional Cascades -> Local Shadow Views -> AO -> Clustered Culling
4. Radiance Lighting
  -> Direct PBR -> Diffuse IBL -> Specular IBL -> Probes -> SSR -> HDR Composition
5. Atmospheric Transport
  -> Sky Background -> Analytic Fog or Foxel Volumetrics
6. Transparent Forward Rendering
7. Camera and Temporal Pipeline
  -> Exposure -> Bloom -> Tone Mapping / Grading -> Temporal Resolve
8. Existing Editor Overlay -> UI -> Present
```

3.3 Ownership architecture

```
Renderer
|-- RenderGraph
|-- GPUScene
|-- ViewportRenderResources
|-- MaterialRegistry / TextureTable
|-- LightRegistry
|-- ShadowSystem
|-- EnvironmentSystem
|-- ReflectionProbeRegistry
|-- ExposureController
|-- TemporalHistoryManager
-- RadiancePipeline (pass registration and orchestration only)
```

Ownership boundary

RadiancePipeline may register and configure passes, but it must not acquire the swapchain, submit queues, own ECS scene state, duplicate GPUScene, create a second scene depth or present frames.

3.4 Shared surface contract

```
struct RadianceSurfaceTargets
{
    ImageHandle depth;
    ImageHandle baseColor;
    ImageHandle normalRoughness;
    ImageHandle materialProperties;
    ImageHandle emissiveShadingModel;
```

```
ImageHandle objectID;
ImageHandle velocity;
};
```

Target	Recommended format	Semantic
depth	D32_SFLOAT	One shared main-view depth.
baseColor	R8G8B8A8_UNORM or reviewed sRGB strategy	Linear surface base color and optional opacity.
normalRoughness	R16G16B16A16_SFLOAT	Documented normal encoding and perceptual roughness.
materialProperties	R8G8B8A8_UNORM	Metallic, material AO and reserved channels.
emissiveShadingModel	R16G16B16A16_SFLOAT preferred	HDR emissive energy and shading model.
objectID	R32_UINT	Entity identity; invalid = 0xFFFFFFFF.
velocity	R16G16_SFLOAT	Current-to-previous screen motion.

3.5 Shared lighting and GPU contracts

```
struct RadianceLightingTargets
{
    ImageHandle directLightingHDR;
    ImageHandle indirectLightingHDR;
    ImageHandle reflectionHDR;
    ImageHandle sceneColorHDR;
};

struct GPULight
{
    Vec4 positionRange;
    Vec4 directionType;
    Vec4 colorIntensity;
    Vec4 spotAnglesSourceRadius;
    uint shadowIndex, layerMask, flags, generation;
};
```

Direct, environment, reflection and emissive contributions must remain individually inspectable even if some are physically composed into one HDR target. Shared GPU structures enter the existing GPUScene upload and descriptor infrastructure.

3.6 Conventional and RVG integration

Conventional Raster -> Standard Surface Targets ---- RVG Visibility -> Attribute Reconstruction / -> Material Resolve -> Standard Surface Targets	-> Shared Radiance Lighting
--	-----------------------------

3.7 Capability behaviour

Capability	Preferred path	Fallback
Descriptor indexing	Bindless tables	Bounded arrays with visible capacity.
Many lights	Clustered compute lists	Capped fullscreen reference loop.
Directional shadows	Stable cascaded atlas	Reduced count/resolution.
Local shadows	Budgeted cached atlas	Highest-priority lights shadowed.
SSR	Hierarchical + temporal	Probe then global environment.

Capability	Preferred path	Fallback
Volumetrics	Froxiel integration	Analytic distance/height fog.
Temporal AA	Motion-vector TAA	FXAA or no-AA.
RVG	Shared surface resolve	Conventional fallback mesh.

4. Global Engineering Rules

4.1 Correctness before optimization

- Do not add a faster path until a clear reference path works.
- Keep the reference light loop after clustered lighting is enabled.
- Do not add temporal accumulation until the current-frame result is stable.
- Do not add auto exposure until manual exposure and HDR are correct.
- Do not bit-pack channels before independent parity tests exist.
- A beautiful final screenshot is not proof that intermediate buffers are correct.

4.2 Integrated-renderer requirement

- Use the current Renderer frame lifecycle and RenderGraph.
- Use one shared scene depth.
- Use one shared material registry and texture table.
- Use GPUScene for persistent records.
- Extend existing viewport resource recreation.
- Preserve transparent, editor overlay, UI and presentation passes.
- Keep unsupported materials and geometry conventional.
- Never implement RVG-specific lighting.

4.3 Ownership rules

- ECS components own lightweight authoring data and stable handles only.
- Registries own CPU metadata and generation-safe slots.
- GPUScene owns persistent GPU records.
- ViewportRenderResources owns viewport-sized images.
- ShadowSystem owns atlas allocation and cache state.
- TemporalHistoryManager owns histories and invalidation.
- No ECS component stores raw Vulkan resources or descriptor indices.
- No serialized asset stores a transient GPU slot or atlas tile.

4.4 Vulkan pass contract

- Declare input/output resources, image layouts, pipeline stages and access masks.
- Declare queue ownership, capacity, overflow, resize and history behaviour.
- Assign a debug name, GPU timestamp and capability fallback to every pass.

```
struct GPUWorkListState
{
    uint count;
    uint capacity;
    uint overflowed;
    uint droppedCount;
    uint highWaterMark;
};
```

4.5 Failure behaviour

Failure	Safe result
Missing material/texture	Visible fallback or documented default.
Shadow atlas exhausted	Light remains active but unshadowed or uses valid lower-resolution cache.
Cluster overflow	Set diagnostics and deterministically drop excess work.
SSR miss/history invalid	Fall back to probe then sky.
No volumetrics	Use analytic fog.
Resize	Recreate safely and invalidate histories.
Unsupported tier	Select lower path and report it.

4.6 Required agent report

- [] Modified/new/deleted files.
- [] Architecture, resource and RenderGraph changes.
- [] Build, runtime and validation results.
- [] Tests and debug views added.
- [] Performance and memory measurements.
- [] Known limitations, blockers and next task.

5. Development Milestones

Milestone	Outcome
I0 Renderer Contract Repair	Trustworthy depth, identity, resize and synchronization.
I1 Shared Surface Contract	Formal GBuffer and geometry convergence semantics.
I2 PBR Material Foundation	Stable materials, textures and BRDF library.
I3 Reference Direct Lighting	Correct light types in linear HDR.
I4 Clustered Lighting	Scalable many-light path proven against reference.
I5 Directional Cascades	Stable sun shadows.
I6 Sky and IBL	HDR sky, irradiance and specular environment.
I7 Ambient Occlusion	Correct contact visibility.
I8 Local Shadows	Budgeted spot/point shadows and caching.
I9 Reflection Probes	Local environment capture.
I10 SSR	Confidence-based screen-space reflections.
I11 Exposure and Color	One-way HDR camera/display pipeline.
I12 Bloom, Velocity and TAA	Stable temporal presentation.
I13 Fog and Volumetrics	Analytic and froxel atmosphere.
I14 Editor and Quality Tiers	Authoring, inspection, profiling and presets.
I15 Production Hardening	Tests, recovery and long-session safety.

Dependency rule

No milestone begins until the exit gates of its dependencies pass. Later effects never justify bypassing an incorrect depth, material, shadow or HDR contract.

6. I0 - Renderer Contract Repair

Objective

Eliminate baseline architecture and Vulkan correctness defects that would invalidate later illumination results.

Dependencies

None.

Architecture integration

Integration boundary

Extends Renderer, RenderSceneExtractor, RadianceFrame, viewport resources and RenderGraph.
Adds no new lighting effect.

Primary deliverables

- Central depth contract
- Dedicated object-ID target
- Stable scene snapshot
- Resource inventory
- Clean baseline validation

Implementation tasks

I0.1 Audit projection and depth

- [] Locate all perspective/orthographic constructors.
- [] Locate GLM depth, handedness and clip-space macros.
- [] Locate all depth clear values and compare operators.
- [] Locate depth reconstruction, HZB and shadow comparisons.
- [] Document standard-Z, reversed-Z and zero-to-one inconsistencies.

I0.2 Define permanent depth convention

```
enum class RadianceDepthConvention
{
    StandardZeroToOne,
    ReversedZeroToOne
};
```

- [] Select StandardZeroToOne or ReversedZeroToOne.
- [] Configure GLM or centralized projection builders for Vulkan zero-to-one depth.
- [] Define near/far policy, clear value, compare operator and invalid background.
- [] Define HZB reduction direction and shadow-depth policy.
- [] Remove ad-hoc pass projection construction.
- [] Add CPU/shader assertions.

I0.3 Repair world-position reconstruction

- [] Add inverseProjection and inverseViewProjection to RadianceFrame.
- [] Calculate inverses once on CPU.
- [] Create shared linear-depth and world-position helpers.
- [] Reject background pixels.
- [] Verify UV/NDC Y orientation and multiplication order.
- [] Add measured-distance, linear-depth and world-position tests.

I0.4 Add dedicated object ID

- [] Create viewport R32_UINT target with attachment, sampled and transfer usage.

- [] Clear to 0xFFFFFFFF.
- [] Add GBuffer unsigned shader output.
- [] Bind it to outline shader and remove shadow-map placeholder.
- [] Update PickEntity to read uint32.
- [] Validate invalid IDs and add integer debug preview.

10.5 Stabilize render-scene snapshot

- [] Make RenderFrame consume a stable snapshot.
- [] Prevent mutable ECS iteration during graph execution.
- [] Define snapshot owner, frame generation and lifetime.
- [] Finalize transforms/bounds before GPUScene upload.

10.6 Formalize viewport ownership

- [] Group viewport images under semantic structures.
- [] Assign one owner for create/resize/descriptors/destruction.
- [] Add generation numbers and deferred destruction.
- [] Invalidate histories after resize/projection changes.
- [] Add Vulkan debug names and resize stress tests.

10.7 Audit RenderGraph synchronization

- [] Declare GBuffer, depth, shadow, AO, HDR, object-ID and final-output dependencies.
- [] Enable synchronization validation.
- [] Replace unjustified ALL_COMMANDS barriers.
- [] Add pass debug labels and timestamps.

10.8 Create resource inventory

- [] Create RADIANCE_FRAME_RESOURCES.md.
- [] Record format, clear, producer, consumers, lifetime and layouts.
- [] Record buffer capacities and frames-in-flight duplication.
- [] Expose generations and ownership in editor.

10 exit gates

Exit gate	Required proof
Depth/reconstruction trustworthy	Measured distances and camera motion pass.
Object identity works	Picking/outline use R32_UINT.
Resize safe	No stale descriptors or validation errors.
Dependencies explicit	Synchronization validation clean.
Resources documented	Runtime inventory matches documentation.

7.11 - Shared Surface Contract

Objective

Turn the existing GBuffer into a permanent semantic interface consumed by all lighting paths and produced by conventional and future RVG resolve.

Dependencies

I0.

Architecture integration

Integration boundary

Introduces semantic surface handles. GBufferPass is the conventional producer; future RVG resolve becomes another producer of the same contract.

Primary deliverables

- RadianceSurfaceTargets
- Depth/space/color docs
- Reviewed formats
- Velocity scaffold
- Surface validation scene

Implementation tasks

I1.1 Audit GBuffer producers/consumers

- [] Locate every writer and reader of GBuffer A-D.
- [] Record normal space, encoding and precision.
- [] Record color transfer handling and channel semantics.
- [] Record clear values and invalid background.
- [] Identify duplicated decode logic.

I1.2 Define RadianceSurfaceTargets

```
struct RadianceSurfaceTargets
{
    ImageHandle depth, baseColor, normalRoughness;
    ImageHandle materialProperties, emissiveShadingModel;
    ImageHandle objectID, velocity;
};
```

- [] Create semantic handles for depth, base color, normal/roughness, material properties, emissive/model, object ID and velocity.
- [] Replace unnamed array indexing with semantic lookup.
- [] Add format checks, resize integration and generation checks.

I1.3 Standardize spaces

- [] Define world/view/local/tangent/clip conventions.
- [] Define handedness, winding and matrix order.
- [] Choose and document GBuffer normal space.
- [] Handle inverse-transpose, negative scale and mirrored tangents.
- [] Add shared CPU/shader conversion helpers.

I1.4 Define color-space contract

- [] Mark color textures sRGB and data textures linear.
- [] Perform material evaluation and lighting in linear HDR.
- [] Choose explicit GBuffer base-color storage semantics.
- [] Prohibit exposure/tonemap in material debug views.

[] Create grayscale/color-patch tests and RADIANCE_COLOR_PIPELINE.md.

I1.5 Select formats

- [] Stress normals, roughness, near-black and emissive values.
- [] Upgrade emissive precision if UNORM clips.
- [] Keep identity integer and avoid premature packing.
- [] Create RADIANCE_TARGET_FORMATS.md.

I1.6 Add velocity scaffold

- [] Allocate R16G16_SFLOAT velocity.
- [] Clear deterministically and add inspection.
- [] Permit zero output until I12.
- [] Mark history invalid when velocity is unavailable.

I1.7 Define conventional/RVG parity

- [] Define required position, normal, tangent, UV, material ID and surface channels.
- [] Document current fallback-mesh convergence.
- [] Define future per-channel tolerances.
- [] Prohibit lighting branches on geometry origin.

I1.8 Build validation scene

- [] Create sphere grid, thin walls, negative/non-uniform scale and imported assets.
- [] Add fixed camera bookmarks.
- [] Add debug and cursor inspection for every channel.
- [] Add deterministic captures and heatmaps.

I1 exit gates

Exit gate	Required proof
Semantics permanent	Every target has encoding, format, clear and owner.
Normals survive transforms	Scale/mirror tests pass.
Color conversion clear	Reference ramps show no double conversion.
Paths can converge	Parity contract includes all lighting inputs.
Debug proof exists	Every channel is viewable and inspectable.

8. I2 - PBR Material Foundation

Objective

Implement stable metallic/roughness material assets, reliable texture resolution and a numerically safe shared BRDF library.

Dependencies

I1.

Architecture integration

Integration boundary

Extends existing material registry, texture table and GBuffer evaluation; adds shared shader includes for deferred and forward paths.

Primary deliverables

- PBRMaterialAsset/GPUMaterial
- Fallback textures
- Texture lifetime protection
- BRDF library
- Material debug panel

Implementation tasks

I2.1 Standard material asset

```
struct GPUMaterial
{
    vec4 baseColorFactor;
    float metallicFactor; roughnessFactor; normalScale; occlusionStrength;
    vec3 emissiveFactor; float emissiveIntensity;
    uint baseColorTexture, normalTexture, metallicRoughnessTexture;
    uint occlusionTexture, emissiveTexture, samplerIndex, flags;
};
```

- [] Separate authored metadata from GPUMaterial.
- [] Create generation-safe MaterialHandle.
- [] Store factors, texture handles, UV transform, sampler, flags and version.
- [] Prohibit serialized descriptor indices.
- [] Route stale handles to fallback.

I2.2 Fallback textures

- [] Create white base color, flat normal, default metallic/roughness, white AO and black emissive.
- [] Create visible error checker for invalid references only.
- [] Ensure data fallbacks are linear.
- [] Test every optional map removed.

I2.3 Texture-table lifetime

- [] Audit global image/sampler tables.
- [] Add slot generation and deferred reuse.
- [] Add bounded lower-tier arrays.
- [] Stress load/unload/hot reload.
- [] Expose capacity and stale-index diagnostics.

I2.4 Material decode helpers

- [] Decode base color in linear space.
- [] Decode/clamp metallic and perceptual roughness.

- [] Convert roughness to GGX alpha.
- [] Decode emissive independently.
- [] Keep alpha semantics separate.

I2.5 Tangent frame and normals

- [] Adopt MikkTSpace-compatible tangents.
- [] Store tangent handedness and re-orthogonalize.
- [] Decode normal maps as data.
- [] Support mirrored UVs, negative and non-uniform scale.
- [] Add import convention metadata.

I2.6 BRDF library

- [] Implement GGX distribution, Smith visibility and Schlick Fresnel.
- [] Use dielectric F0 near 0.04 and metal base-color F0.
- [] Implement Lambert/Burley diffuse and energy conservation.
- [] Create EvaluateDirectBRDF.
- [] Protect denominators and add NaN/Inf tests.

I2.7 Emissive

- [] Decode HDR emissive factor/intensity.
- [] Add after reflected lighting.
- [] Do not multiply by NdotL, shadows or AO.
- [] Keep visible without lights.

I2.8 Shared material resolve

- [] Use one GPUMaterial for both geometry paths.
- [] Resolve slot + instance override to final material ID.
- [] Route unsupported RVG materials conventional.
- [] Compare channels before final-lit parity.

I2.9 Validation and inspection

- [] Create metallic/roughness sphere captures.
- [] Add plastic, metal, seams, emissive and calibrated patches.
- [] Test reload/deletion.
- [] Add base color, roughness, metallic, AO, emissive, F0, tangent and UV modes.

I2 exit gates

Exit gate	Required proof
Assets stable	Reload/deletion resolves valid data or fallback.
Sphere grid coherent	Sweeps are smooth and firefly-free.
Normal mapping correct	Mirrored UV and transform tests pass.
BRDF safe	No NaN/Inf or uncontrolled spikes.
One material system	Both geometry paths share records and decode.

9. I3 - Reference Direct Lighting

Objective

Produce correct directional, point and spot direct lighting through a deliberately simple reference deferred pass before many-light optimization.

Dependencies

I2.

Architecture integration

Integration boundary

Refactors current DeferredLightingPass into a clear reference PBR responsibility using GPUScene lights and shared surfaces.

Primary deliverables

- Light registry/GPULight
- Reference deferred pass
- Three light types
- Light gizmos/diagnostics

Implementation tasks

I3.1 Standardize light components

- [] Audit directional/point/spot ECS components.
- [] Define intensity units or engine scale.
- [] Add RGB/Kelvin, range, source radius, cone angles, layer mask and shadow flag.
- [] Use range as culling boundary.
- [] Serialize authoring data only.

I3.2 LightRegistry

- [] Allocate stable generation-checked slots.
- [] Map entity to slot and reverse debug lookup.
- [] Upload dirty records only.
- [] Maintain compact active index list.
- [] Defer slot reuse.

I3.3 ReferenceDeferredLightingPass

- [] Read shared surface targets.
- [] Reconstruct position and reject background.
- [] Loop a capped local-light list and separate directional loop.
- [] Disable shadows/IBL initially.
- [] Lock exposure and write linear HDR.
- [] Add CPU pixel comparison and keep pass permanently.

I3.4 Directional evaluation

- [] Define direction sign.
- [] Transform to shading space.
- [] Evaluate shared BRDF.
- [] Test world-fixed lighting under camera rotation.

I3.5 Point attenuation

- [] Use inverse-square attenuation with close-distance safety.
- [] Apply smooth range window.
- [] Measure luminance ratios at known distances.

[] Prevent near-source explosions.

I3.6 Spot attenuation

- [] Convert inner/outer angles to cosines.
- [] Apply smooth cone interpolation times point attenuation.
- [] Validate cosine ordering.
- [] Add cone gizmo and boundary test.

I3.7 Layer masks

- [] Add masks to light and instance records.
- [] Reject non-overlap.
- [] Keep universal default.
- [] Add isolation test.

I3.8 Diagnostics

- [] Add light-type, direct diffuse and direct specular views.
- [] Inspect radiance, attenuation, NdotL and F0 under cursor.
- [] Record evaluated-light count and GPU time.

I3 exit gates

Exit gate	Required proof
Light types correct	Reference matches CPU-selected pixels.
Attenuation predictable	Distance/cone tests monotonic.
Linear HDR preserved	No tone-map/gamma/clamp in pass.
Light slots stable	No stale lights during create/delete/mode changes.
Reference retained	Can switch back for later parity.

10. I4 - Clustered Lighting Production Path

Objective

Harden the existing clustered-light culling and shading path so it scales to many lights while remaining equivalent to I3.

Dependencies

I3 and stable depth.

Architecture integration

Integration boundary

Upgrades existing LightCullingPass and DeferredLightingPass; no independent many-light renderer.

Primary deliverables

- Shared cluster mapping
- Bounded lists
- Parity mode
- Complexity profiler
- Tier fallback

Implementation tasks

I4.1 Audit cluster resources

- [] Record grid dimensions and depth slicing.
- [] Verify CPU/shader layouts for bounds, ranges, indices and settings.
- [] Verify culling and shading share the same mapping.
- [] Record capacities, clears and current overflow behaviour.

I4.2 Shared ClusterMapping

- [] Centralize screen tile and depth-slice mapping.
- [] Use logarithmic/hybrid depth slices.
- [] Protect near-plane and invalid-depth cases.
- [] Add boundary unit tests.

I4.3 Harden list generation

- [] Reset counters every frame.
- [] Conservatively intersect light sphere/cone with clusters.
- [] Apply layer masks.
- [] Guard every append.
- [] Record overflow, dropped count and high-water mark.
- [] Use deterministic drop order.
- [] Insert compute-to-lighting barriers.

I4.4 Clustered shading

- [] Select cluster from pixel/depth.
- [] Loop only its light range.
- [] Validate light generations.
- [] Use identical BRDF and attenuation helpers as reference.
- [] Keep directional lights separate.

I4.5 Parity comparison

- [] Render reference and clustered paths for identical frames.
- [] Add split-screen, alternating-frame and difference views.

- [] Test overlaps, depth boundaries, masks and viewport edges.
- [] Record mean/max error and enforce tolerance.

I4.6 Complexity/profiling

- [] Visualize lights per cluster and per pixel.
- [] Show average/max/percentile occupancy and overflow clusters.
- [] Record culling/list/shading GPU timings.
- [] Stress 10, 100 and 1000 lights.

I4.7 Lower-tier fallback

- [] Retain capped reference loop.
- [] Define fallback light count.
- [] Auto-select when compute/storage features unavailable.
- [] Report active path and reject unsupported forced modes.

I4 exit gates

Exit gate	Required proof
Matches reference	Difference captures within tolerance.
Mapping stable	No lights disappear at boundaries.
Overflow safe	Stress reports without memory corruption.
Cost measurable	Timings and occupancy visible.
Fallback works	Lower tier still produces valid lighting.

11. I5 - Directional Shadow Cascades

Objective

Replace the single sun shadow with stable cascaded views, atlas ownership, culling, bias, filtering and diagnostics.

Dependencies

I0, I3 and depth-only geometry.

Architecture integration

Integration boundary

Generalizes existing ShadowPass into common render views and an atlas while reusing geometry and light data.

Primary deliverables

- RadianceViewData
- Cascade calculator
- Directional atlas
- Depth-only pipeline
- PCF/blending/diagnostics

Implementation tasks

I5.1 Generalize RadianceViewData

- [] Represent main camera and shadow views in one structure.
- [] Store matrices, frustum, atlas rectangle, layer mask, view ID and LOD threshold.
- [] Remove main-camera assumptions from depth-only submission.

I5.2 Practical cascade splits

- [] Add cascade count and shadow distance.
- [] Blend uniform/log splits with lambda.
- [] Store splits in view space.
- [] Use identical shader convention.
- [] Add cascade-band view.

I5.3 Frustum corners

- [] Generate eight world-space corners per slice.
- [] Support perspective and orthographic cameras.
- [] Avoid ambiguous NDC assumptions.
- [] Draw cascade frusta.

I5.4 Light-space fit

- [] Build directional-light basis.
- [] Fit conservative orthographic bounds.
- [] Expand depth for off-screen casters.
- [] Test tall off-screen caster.

I5.5 Texel stabilization

- [] Calculate world units per texel.
- [] Snap center/bounds before final projection.
- [] Keep extents stable.
- [] Compare stabilized/unstabilized motion.

I5.6 Directional atlas

- [] Allocate padded cascade tiles.
- [] Store scale/offset, resolution, generation and owner.
- [] Clear safely.
- [] Clamp PCF to tile.
- [] Reallocate safely after quality changes.

I5.7 Depth-only pipeline

- [] Use minimal shadow pipeline.
- [] Bind shared geometry/GPUScene.
- [] Support conventional and RVG fallback.
- [] Handle mirrored winding.
- [] Reserve alpha-mask path.

I5.8 Per-cascade culling

- [] Build view-specific caster lists.
- [] Do not depend on main-view visibility.
- [] Apply cast-shadow flags/masks.
- [] Record draw/triangle counts.
- [] Define future RVG coarse thresholds.

I5.9 Bias controls

- [] Add constant and slope bias.
- [] Add receiver normal offset using geometric normal.
- [] Expose safe defaults.
- [] Create acne/Peter-Panning tests.

I5.10 Select/filter/blend

- [] Select by view-space depth.
- [] Implement 3x3 PCF in texel units.
- [] Prevent atlas leakage.
- [] Blend adjacent cascades in transition bands.
- [] Test moving boundaries.

I5.11 Future RVG priority

- [] Define main/cascade view masks.
- [] Prioritize main view.
- [] Allow distant cascades to use resident parents.
- [] Deduplicate requests and prohibit holes.

I5.12 Diagnostics

- [] Show atlas, cascade index, splits, receiver/sample depth, bias and PCF.
- [] Show caster path and per-cascade timings.
- [] Add freeze controls and regression cases.

I5 exit gates

Exit gate	Required proof
Stable sun shadows	Slow camera motion has no severe swimming.
Coverage correct	Off-screen casters and receiver slices pass.
Bias controlled	No major acne or detachment.
Transitions unobtrusive	No visible boundary rings.
Atlas safe	No cross-tile samples or stale owners.

Exit gate	Required proof
Cost visible	Per-cascade workload/timing reported.

12. I6 - Sky and Image-Based Lighting

Objective

Add a visible HDR sky and diffuse/specular environment illumination so shadows remain readable and metals reflect the world.

Dependencies

I2, I3 and linear HDR.

Architecture integration

Integration boundary

Adds SkyLight/EnvironmentSystem plus sky and environment passes using existing HDRColor and post-processing.

Primary deliverables

- SkyLight
- HDR importer/cache
- SkyBackgroundPass
- Irradiance/prefilter/LUT
- EnvironmentLightingPass
- Manual exposure

Implementation tasks

I6.1 SkyLight and environment asset

- [] Store environment handle, rotation, diffuse/specular intensity and exposure offset.
- [] Support HDR/procedural modes.
- [] Register one global SkyLight initially.
- [] Serialize authoring data only.

I6.2 HDR import

- [] Support HDR/EXR.
- [] Preserve linear values above one.
- [] Convert equirectangular to cubemap.
- [] Record orientation/flip metadata.
- [] Generate mips and cache processed assets.
- [] Use neutral fallback on failure.

I6.3 Sky background

- [] Render only at invalid depth.
- [] Use camera rotation without translation.
- [] Do not write depth or object ID.
- [] Keep background display separate from lighting.

I6.4 Diffuse irradiance

- [] Convolve environment over hemisphere.
- [] Store low-resolution irradiance cube or validated SH.
- [] Cache by source/settings.
- [] Validate diffuse sphere response.

I6.5 Specular prefilter and LUT

- [] Generate GGX-prefiltered cubemap mips.
- [] Map perceptual roughness consistently.
- [] Generate BRDF integration LUT.

[] Validate roughness progression.

16.6 EnvironmentLightingPass

- [] Read surface/material/view data.
- [] Evaluate diffuse irradiance for non-metals.
- [] Evaluate prefiltered specular with Fresnel and BRDF LUT.
- [] Write inspectable indirect HDR.
- [] Do not use flat ambient color.

16.7 Manual exposure

- [] Define documented exposure/EV scale.
- [] Store exposure in RadianceFrame.
- [] Apply once before tone map.
- [] Add lock/reset.
- [] Keep auto exposure disabled.

16.8 Initial filmic transform

- [] Choose documented tone-map operator.
- [] Apply only in post-process.
- [] Avoid double gamma.
- [] Add neutral/filmic modes and grayscale/highlight tests.

16.9 Diagnostics

- [] Preview source, irradiance, prefilter mips and BRDF LUT.
- [] Show diffuse/specular IBL separately.
- [] Inspect HDR texels.
- [] Record preprocessing time/memory.

16 exit gates

Exit gate	Required proof
Infinite sky	Translation does not move sky; rotation does.
HDR energy present	Raw values/reflections exceed one.
Shadows readable	Irradiance lights without flat ambient.
Metals reflect world	Roughness/rotation behave smoothly.
Display conversion singular	Exposure/tone-map/transfer each occur once.

13. I7 - Ambient Occlusion

Objective

Correct and evolve screen-space ambient visibility without treating AO as a universal final-color darkener.

Dependencies

I1, stable depth and I6.

Architecture integration

Integration boundary

Extends existing SSAOPass/AOBlurPass and composes AO inside environment lighting.

Primary deliverables

- Correct SSAO
- Bilateral filter/upsample
- Optional GTAO
- AO composition contract
- AO debug views

Implementation tasks

I7.1 Audit existing SSAO

- ☐ Verify depth linearization and normal space.
- ☐ Verify radius/bias scale, projection and noise.
- ☐ Reject background.
- ☐ Inspect edge-aware blur.
- ☐ Capture baseline.

I7.2 Deterministic SSAO reference

- ☐ Use fixed hemisphere kernel.
- ☐ Reconstruct view-space positions.
- ☐ Use deterministic rotation.
- ☐ Apply thickness/bias.
- ☐ Output visibility in $[0,1]$.
- ☐ Keep temporal jitter disabled.

I7.3 Bilateral filter/upsample

- ☐ Filter with depth and normal similarity.
- ☐ Prevent silhouette bleeding.
- ☐ Support full/half resolution.
- ☐ Add depth-aware upsample.

I7.4 GTAO upgrade

- ☐ Implement horizon sampling where practical.
- ☐ Calculate ambient visibility with thickness compensation.
- ☐ Compare to SSAO.
- ☐ Keep SSAO fallback.

I7.5 Temporal AO

- ☐ Reproject with velocity when valid.
- ☐ Reject by depth/normal mismatch.
- ☐ Clamp history.
- ☐ Reset on resize/cut/projection/quality.
- ☐ Expose rejection mask.

I7.6 Correct composition

- [] Apply AO to diffuse environment lighting.
- [] Add controlled specular occlusion.
- [] Do not darken direct or emissive.
- [] Do not bake AO into base color.

I7.7 Diagnostics/tests

- [] Show raw, filtered and temporal AO.
- [] Create corner, thin-wall, contact and near-plane scenes.
- [] Measure AO/blur cost.
- [] Add deterministic captures.

I7 exit gates

Exit gate	Required proof
Ambient visibility correct	Directly lit surfaces not wrongly black.
Contact without halos	Corners improve; silhouettes remain clean.
Conventions match	Camera motion shows no projection error.
Temporal safe	No severe trails; resets work.
Fallback retained	Reference SSAO remains available.

14. I8 - Local-Light Shadows

Objective

Add budgeted spot and point shadows through a managed atlas, importance scheduler, cache invalidation and explicit quality budgets.

Dependencies

I4 and I5.

Architecture integration

Integration boundary

Extends common ShadowSystem and view contracts; local lights remain in the same LightRegistry and direct-lighting path.

Primary deliverables

- Variable atlas allocator
- Importance scheduler
- Spot/point shadows
- Cache/invalidation
- Budgets/diagnostics

Implementation tasks

I8.1 ShadowAtlasAllocator

- [] Support variable power-of-two tiles.
- [] Store generation, owner, padding, last-used frame and cache state.
- [] Choose buddy/skyline/fixed-bin allocation.
- [] Defer reuse until frames complete.
- [] Track occupancy, fragmentation and failures.
- [] Prevent PCF crossing tiles.

I8.2 Importance scoring

- [] Combine projected area, intensity, distance, selection/gameplay priority and recent visibility.
- [] Map score to resolution/update frequency.
- [] Add deterministic ties and hysteresis.
- [] Expose score components.

I8.3 Spot shadow views

- [] Build perspective view from position/direction.
- [] Use outer cone plus guard band.
- [] Choose near plane from source size and scale.
- [] Set far plane to range.
- [] Map into atlas tile and draw frustum gizmo.

I8.4 Spot render/sample

- [] Cull casters to spot frustum.
- [] Render common shadow depth.
- [] Project receiver and reject outside cone.
- [] Apply bias and PCF.
- [] Test thin walls and moving casters.

I8.5 Point representation

- [] Use six 90-degree faces for reference.
- [] Define face orientation constants.

- [] Choose cubemap array or six tiles.
- [] Document radial depth.
- [] Keep octahedral as later optimization.

l8.6 Point render/sample

- [] Cull per face.
- [] Store documented depth/radial value.
- [] Sample by light-to-receiver direction.
- [] Apply consistent comparison, bias and filtering.
- [] Test rotation invariance and seams.

l8.7 Shadow caching

- [] Define static, stationary and dynamic classes.
- [] Cache unchanged static depth.
- [] Track light/caster/material dependencies.
- [] Record cache age and last render.

l8.8 Selective invalidation

- [] Update bounds before queries.
- [] Intersect changed casters with light volumes.
- [] Dirty only matching caches.
- [] Record reason/entity.
- [] Handle unload/reload and cast-shadow changes.

l8.9 Quality budgets

- [] Set atlas size, resolution tiers, max shadowed lights and updates/frame.
- [] Set point-face budget and PCF quality.
- [] Switch tiers safely.
- [] Do not alter authored light energy.

l8.10 Exhaustion policy

- [] Keep light active if allocation fails.
- [] Use unshadowed visibility or valid lower cache.
- [] Never alias live tiles.
- [] Emit rate-limited diagnostics.
- [] Stress tiny atlas.

l8.11 Diagnostics

- [] Show atlas owners and cube faces.
- [] Display resolution, cache age, dirty reason, caster count and GPU time.
- [] Show denied allocation reason and fallback.

l8 exit gates

Exit gate	Required proof
Spot shadows correct	Cone/frustum and silhouette tests pass.
Point coverage complete	Six-axis tests show no missing faces.
Caching saves work	Static shadows stop updating.
Invalidation selective	Only intersecting lights refresh.
Budget safe	Denied shadows remain valid unshadowed lights.

15. I9 - Reflection Probes

Objective

Add local environment captures for indoor/spatially varying reflections and provide a robust fallback for SSR.

Dependencies

I6.

Architecture integration

Integration boundary

Adds ReflectionProbeRegistry and capture/resolve paths feeding the existing specular environment lighting.

Primary deliverables

- Probe component/registry
- Static cubemap capture
- Prefiltered assets
- Probe resolve/parallax correction
- Probe authoring

Implementation tasks

I9.1 Probe component/registry

- [] Support sphere and box influence.
- [] Store capture position, priority, blend distance, update mode and parallax bounds.
- [] Allocate generation-safe slots.
- [] Serialize authoring data only.

I9.2 Static capture

- [] Render six HDR cubemap faces.
- [] Exclude editor overlays and the probe itself.
- [] Use exposure-independent HDR lighting.
- [] Generate mips and roughness prefilter.
- [] Store scene version/checksum.

I9.3 Influence resolution

- [] Find affecting probes.
- [] Select/blend by priority and normalized distance.
- [] Cap probe count.
- [] Fall back to global sky.
- [] Add deterministic overlap and index/weight views.

I9.4 Box parallax correction

- [] Intersect reflection ray with box bounds.
- [] Build corrected lookup direction.
- [] Protect outside/edge cases.
- [] Compare corrected/uncorrected rooms.

I9.5 Integrate environment lighting

- [] Use local probe for specular when valid.
- [] Blend to global sky at boundaries.
- [] Keep diffuse irradiance global initially.
- [] Use same roughness and BRDF LUT.

l9.6 Authoring/diagnostics

- [] Add probe creation, gizmos, capture preview and bake controls.
- [] Show faces/mips, capture age, source version and memory.
- [] Create indoor/outdoor/overlap tests.

l9 exit gates

Exit gate	Required proof
Spatial variation works	Expected probe selected indoors/outdoors.
Blending stable	No hard reflection pops.
Parallax improves rooms	Corrected reflection aligns better.
Global fallback guaranteed	No black reflection when probes absent.
Capture reproducible	Locked scenes recapture deterministically.

16. I10 - Screen-Space Reflections

Objective

Add visible-scene reflections using confidence and an explicit fallback chain to probes and sky.

Dependencies

I1, I9 and stable HDR.

Architecture integration

Integration boundary

Adds SSR after opaque HDR lighting with probes and sky as mandatory fallback providers.

Primary deliverables

- Shared HZB
- Linear reference SSR
- Hierarchical SSR
- Confidence target
- Temporal/fallback blend

Implementation tasks

I10.1 Input contract

- [] Require depth, normal, roughness, HDR scene and camera matrices.
- [] Require HZB or create shared depth pyramid.
- [] Use velocity/history only for temporal stage.
- [] Reject background/invalid/very rough pixels.
- [] Document sampled scene-color stage.

I10.2 Shared HZB

- [] Create mipmapped depth using permanent convention.
- [] Generate correct reduction mips and barriers.
- [] Handle resize/cut invalidation.
- [] Share with RVG only if semantics/timing match.
- [] Add mip viewer/freeze.

I10.3 Linear reference SSR

- [] Reconstruct view position/reflection direction.
- [] March with fixed max steps.
- [] Compare ray depth using thickness tolerance.
- [] Refine hit.
- [] Reject screen/behind-camera misses.
- [] Output HDR color and confidence.

I10.4 Hierarchical SSR

- [] Traverse HZB to skip empty intervals.
- [] Select mip from footprint.
- [] Descend/refine.
- [] Protect near-plane and edges.
- [] Add roughness-dependent distance/quality.
- [] Compare with reference.

I10.5 Confidence

- [] Combine depth error, edge distance, ray length, grazing angle and roughness.
- [] Reject low confidence.

- [] Store dedicated confidence target.
- [] Expose components under cursor.

I10.6 Temporal SSR

- [] Reproject reflection/confidence.
- [] Validate depth, normal and hit consistency.
- [] Reject disocclusion.
- [] Clamp history.
- [] Handle exposure if needed.
- [] Reset on cut/resize/projection.

I10.7 Fallback hierarchy

- [] Use high-confidence SSR.
- [] Blend to resolved probe as confidence falls.
- [] Fall back to global sky.
- [] Avoid black edge holes.
- [] Preserve roughness filtering.

I10.8 Diagnostics/tests

- [] Show ray steps, hit UV, depth error, confidence and source.
- [] Create mirror, rough floor, thin object, off-screen and cut tests.
- [] Measure reference/hierarchical/temporal cost.

I10 exit gates

Exit gate	Required proof
Plausible hits	Smooth surfaces reflect visible geometry.
Graceful misses	Edges/off-screen blend to probes/sky.
Hierarchy matches reference	Difference within tolerance.
Temporal controlled	No severe trails; cuts reset.
HZB shared safely	Documented compatible semantics.

17. I11 - Exposure, Tone Mapping and Color

Objective

Formalize a one-way transformation from linear HDR scene radiance to display output, including manual and automatic exposure.

Dependencies

I6 and stable HDR composition.

Architecture integration

Integration boundary

Expands existing PostProcessPass into explicit luminance, exposure, tone-map and grading stages while Renderer retains final-output ownership.

Primary deliverables

- Camera pipeline doc
- Luminance histogram
- Auto exposure
- Final tone mapper
- Grading/diagnostics

Implementation tasks

I11.1 Document camera/color pipeline

- [] Define linear working space.
- [] Define exposure, tone-map, grading and display-transfer order.
- [] Define swapchain sRGB policy.
- [] Prohibit earlier gamma/tone-map.
- [] Create RADIANCE_CAMERA_PIPELINE.md.

I11.2 Luminance measurement

- [] Calculate log luminance.
- [] Reject invalid pixels by policy.
- [] Implement histogram or reduction.
- [] Add percentile clipping.
- [] Store compact exposure statistics.
- [] Add histogram/log-luminance views.

I11.3 Automatic exposure

- [] Convert luminance to target exposure/EV.
- [] Add min/max and compensation.
- [] Add separate adaptation speeds.
- [] Use stable delta-time adaptation.
- [] Handle camera cuts.
- [] Keep manual/locked modes.
- [] Store current/previous exposure.

I11.4 Final tone mapping

- [] Select default filmic operator and neutral reference.
- [] Add white-point/shoulder controls only where meaningful.
- [] Preserve hue through highlights.
- [] Test near-black and extreme lights.
- [] Apply once after HDR effects.

I11.5 Color grading

- [] Add white balance/tint, contrast, saturation, lift/gamma/gain.
- [] Define grading order/space.
- [] Defer LUT until scalar controls stable.
- [] Version/serialize settings.

I11.6 Display validation

- [] Ensure output encoded once.
- [] Compare editor, runtime and screenshots.
- [] Test sRGB/non-sRGB formats.
- [] Add grayscale/color-patch captures.
- [] Add double/missing gamma debug.

I11.7 Diagnostics

- [] Show current/target exposure and histogram percentiles.
- [] Show pre-exposure HDR, exposed HDR, tone-mapped and final output.
- [] Add freeze/override/frame-step.
- [] Record timings.

I11 exit gates

Exit gate	Required proof
HDR preserved	Lighting/atmosphere remain linear.
Exposure predictable	Manual/auto reach documented results.
Adaptation stable	Indoor/outdoor transitions smooth.
Tone-map singular	No duplicate compression/gamma.
Viewport/capture agree	Same display transform used.

18. I12 - Bloom, Velocity and Temporal Anti-Aliasing

Objective

Add HDR bloom and a correct motion-vector temporal pipeline that stabilizes edges and histories without hiding current-frame errors.

Dependencies

I1, I11 and stable transforms.

Architecture integration

Integration boundary

Adds velocity to shared surface output, a central history manager, bloom and temporal resolve before editor overlays.

Primary deliverables

- Velocity implementation
- Projection jitter
- TemporalHistoryManager/TAA
- Bloom chain
- FXAA/no-AA

Implementation tasks

I12.1 Current/previous transforms

- ☐ Store current/previous object transforms in GPUScene.
- ☐ Store current/previous camera view-projection.
- ☐ Preserve previous before update.
- ☐ Handle new, teleported and destroyed objects.
- ☐ Handle edit/play cloning.
- ☐ Add camera-cut/projection flags.

I12.2 Per-pixel velocity

- ☐ Calculate current/previous clip positions.
- ☐ Define screen-space convention.
- ☐ Handle new/disoccluded pixels.
- ☐ Support camera-only and object-only motion.
- ☐ Support conventional and future RVG.
- ☐ Add velocity visualization.

I12.3 Projection jitter

- ☐ Choose deterministic low-discrepancy sequence.
- ☐ Apply only in temporal modes.
- ☐ Store current/previous jitter.
- ☐ Use unjittered or documented matrices for culling/shadows.
- ☐ Disable for deterministic captures.

I12.4 Temporal resolve

- ☐ Reproject with velocity.
- ☐ Validate depth, normal and object identity.
- ☐ Detect disocclusion.
- ☐ Clamp history to neighbourhood/variance.
- ☐ Add responsive handling for emissive/thin geometry.
- ☐ Choose documented HDR/display history space.
- ☐ Reset on resize/cut/FOV/quality.

I12.5 Bloom

- [] Extract bright HDR energy with threshold/knee.
- [] Build downsample pyramid.
- [] Filter/upsample and composite pre-tone-map.
- [] Prevent NaN/Inf contamination.
- [] Expose each level and cost.

I12.6 FXAA/no-AA

- [] Implement FXAA fallback and no-AA debug.
- [] Disable jitter when TAA inactive.
- [] Invalidate histories on mode switch.
- [] Report active AA path.

I12.7 TemporalHistoryManager

- [] Register AO, SSR, volume and final-color histories.
- [] Centralize reset reasons.
- [] Protect viewport generations.
- [] Track memory/last valid frame.
- [] Add freeze/clear controls.

I12.8 Diagnostics/tests

- [] Show current, history, velocity, disocclusion, clamp bounds and weight.
- [] Create rotating, panning, camera-cut, thin and emissive tests.
- [] Measure shimmer/ghosting and timings.

I12 exit gates

Exit gate	Required proof
Velocity correct	Camera/object motion direction and magnitude pass.
Temporal stable	Static edges improve without severe trails.
Invalidation works	Resize/cuts use current frame and rebuild.
Bloom uses HDR	Bright emission blooms before tone-map.
Fallback complete	FXAA/no-AA produce valid output without jitter.

19. I13 - Fog and Volumetric Lighting

Objective

Add atmospheric depth as participating-media transport, starting with analytic fog and progressing to froxel volumetrics.

Dependencies

I5, I11 and I12.

Architecture integration

Integration boundary

Adds atmosphere components and analytic/froxel passes that composite into HDR before final camera presentation.

Primary deliverables

- Fog/atmosphere components
- Distance/height fog
- Froxel resources/injection
- Volumetric integration/history
- Atmosphere diagnostics

Implementation tasks

I13.1 Atmosphere authoring

- [] Create global atmosphere settings, HeightFog and optional FogVolume.
- [] Store density, extinction/scattering, height falloff and anisotropy.
- [] Version/serialize authoring only.
- [] Add capability fallback settings.

I13.2 Distance fog

- [] Reconstruct distance.
- [] Calculate exponential transmittance.
- [] Composite $T \cdot \text{scene} + \text{in-scatter}$.
- [] Handle sky/background.
- [] Keep as lowest-tier fallback.

I13.3 Height fog

- [] Define base height and exponential falloff.
- [] Approximate ray integration.
- [] Handle camera above/inside/below layer.
- [] Avoid double extinction.
- [] Add density/transmittance views.

I13.4 Froxel grid

- [] Choose XY tiles and logarithmic/hybrid Z slices.
- [] Reuse clustered slicing where compatible.
- [] Allocate extinction, scattering and integrated volumes.
- [] Handle resize/history.
- [] Define memory budgets and slice viewer.

I13.5 Medium injection

- [] Evaluate global and local density.
- [] Resolve overlapping volumes.
- [] Store extinction/scattering.

- [] Guard local-volume lists.
- [] Add density debug.

I13.6 Light injection

- [] Add directional scattering and cascaded-shadow sampling.
- [] Add selected clustered local lights.
- [] Use documented phase function/anisotropy.
- [] Budget shadowed local volumetric lights.

I13.7 Front-to-back integration

- [] Accumulate in-scatter/transmittance.
- [] Use stable numeric integration.
- [] Composite into HDR at documented order.
- [] Define sky/transparency interaction.

I13.8 Temporal reprojection

- [] Reproject volume history.
- [] Reject cuts/resize/projection changes.
- [] Clamp unstable history.
- [] Add jitter and show reset reason.
- [] Prevent ghost shafts.

I13.9 Diagnostics/quality

- [] Show froxel slices, density, extinction, scattering and shadows.
- [] Display resolution/memory/time.
- [] Add analytic/froxel comparison and quality presets.
- [] Create corridor, shaft and local-light tests.

I13 exit gates

Exit gate	Required proof
Analytic fallback correct	Fog increases smoothly and preserves depth.
Light visibility respected	Shafts/local scatter respond to shadows.
Order documented	Sky/opaque/volume/transparency combine predictably.
Temporal controlled	No severe flicker or persistent shafts.
Quality safe	Disabling froxels selects analytic fog.

20. I14 - Editor Authoring, Diagnostics and Quality Tiers

Objective

Make every feature authorable, inspectable, profileable and scalable inside Omnix Studio without serializing transient GPU details.

Dependencies

I0-I13.

Architecture integration

Integration boundary

Extends existing editor panels, render-target preview and profiler; adds a consolidated Radiance panel, not a separate application.

Primary deliverables

- Serializable components
- Post-process stack
- Debug panel/inspector
- Profiler/memory telemetry
- Quality/capability presets

Implementation tasks

I14.1 ECS authoring components

- [] Version/serialize lights, SkyLight, ReflectionProbe, HeightFog and FogVolume.
- [] Create PostProcessVolume or equivalent.
- [] Store stable handles and authored properties only.
- [] Do not serialize GPU indices, atlas tiles or history.
- [] Add scene migration.

I14.2 Post-process volume resolution

- [] Support global/bounded volumes.
- [] Store priority, blend radius/weight and per-property overrides.
- [] Resolve camera stack deterministically.
- [] Blend exposure, grading, bloom and fog.
- [] Inspect resolved stack.
- [] Reset affected histories on discontinuities.

I14.3 Radiance panel

- [] Add Overview, Surface, Materials, Lights, Shadows, Environment, AO, Reflections, Exposure, Temporal, Atmosphere, Resources and Profiling tabs.
- [] Show active path/tier, capacities, overflows and high-water marks.
- [] Add safe overrides and freeze controls.

I14.4 Resource inspector

- [] Preview 2D images, arrays, cubemap faces and volume slices.
- [] Support channels, integer palettes, normals, linear depth and mips.
- [] Show cursor values, format, extent, generation, layout and producer.

I14.5 Profiling telemetry

- [] Timestamp every pass.
- [] Handle delayed query readback.
- [] Show current/average/min/max/percentiles.
- [] Record workload context and CSV.

[] Track CPU preparation separately.

I14.6 Memory/capacity

- [] Track GBuffer, shadow atlas, descriptor tables, cluster lists, probes, HZB, histories, volume and upload memory.
- [] Report high-water marks and overflows.
- [] Warn before silent degradation.

I14.7 Quality presets

- [] Create Low/Medium/High/Cinematic.
- [] Configure cascades, local shadows, AO, SSR, bloom, TAA and volumes.
- [] Keep authored light/material values unchanged.
- [] Support project defaults/runtime overrides.

I14.8 Capability fallbacks

- [] Select clustered/reference, local-shadow budgets, SSR/probes, froxel/analytic fog and TAA/FXAA.
- [] Generate startup report.
- [] Allow forced mode only if supported.

I14.9 Authoring validation

- [] Warn on excessive light budgets, probe overlap/missing captures, absent SkyLight, conflicting volumes and unstable fog/exposure.
- [] Provide locate-entity commands.

I14 exit gates

Exit gate	Required proof
All authorable	Components save/reload correctly.
Every result inspectable	Major resources/debug modes exist.
Cost/capacity visible	Timings and bounded usage reported.
Presets semantic	Quality changes algorithms, not authored energy.
Fallback transparent	Reports explain active paths.

21. I15 - Production Hardening

Objective

Make Radiance safe for long editor sessions, reload, capacity exhaustion, device recreation and production content.

Dependencies

I0-I14.

Architecture integration

Integration boundary

Hardens all integrated subsystems while Renderer remains owner of frame lifecycle, recreation and presentation.

Primary deliverables

- Failure recovery matrix
- Hot reload/recreation
- Automated tests
- Golden baselines
- Stress tools/telemetry
- Final report

Implementation tasks

I15.1 Failure recovery

- [] Handle missing/corrupt materials, textures and environments.
- [] Handle stale/exhausted shadow allocations.
- [] Handle cluster, SSR, probe and volume overflow.
- [] Handle invalid histories and pipeline failures.
- [] Guarantee reduced quality/diagnostic rather than crash, hang or black frame.

I15.2 Hot reload

- [] Reload materials, textures, environments and shaders safely.
- [] Keep old resources until replacement ready.
- [] Increment generations and update descriptors.
- [] Invalidate processed caches/histories.
- [] Defer destruction.

I15.3 Device/renderer recreation

- [] Recreate viewport targets, atlases, environment/probes, HZB, descriptors, pipelines and queries.
- [] Restore registries from CPU metadata.
- [] Reset histories/exposure.
- [] Produce fallback-lit frame during restoration.

I15.4 Unit tests

- [] Test handles/generations, depth reconstruction, BRDF, cascade splits/snapping, atlas allocator, cluster mapping, color transforms, temporal invalidation and serialization.

I15.5 GPU/integration tests

- [] Test GBuffer, clustered parity, shadow atlas/cascades, AO/SSR/volume lists, scene load/unload, resize, teleport, hot reload, low-tier startup and large supported workloads.

I15.6 Golden images

- [] Capture surface, direct/indirect light, shadows, AO, reflections, exposure and atmosphere.
- [] Lock camera, jitter, time and exposure.

- [] Warm/freeze temporal state.
- [] Generate heatmaps and vendor tolerances.

I15.7 Failure injection

- [] Simulate tiny atlases/tables, forced overflow, missing resources, invalid histories/pipelines, slow preprocessing and failed loads.
- [] Run editor soak tests.
- [] Verify safe degradation.

I15.8 Telemetry baselines

- [] Record scene extraction, GPUScene upload, every pass timing, memory and workload.
- [] Create development/target GPU baselines.
- [] Export milestone reports/CSV and regression thresholds.

I15.9 Final documentation

- [] Update architecture, resource, depth, color, capability, pass ordering, descriptors, quality/fallback and editor docs.
- [] Record non-goals/deferred features.
- [] Produce final completion report.

I15 exit gates

Exit gate	Required proof
Extended sessions safe	Long editor/resize/scene tests pass.
Hot reload safe	Old resources stay valid until replacement.
Recreation works	Core lighting returns without invalid state.
Exhaustion graceful	Stress reduces quality, not correctness.
Automated validation passes	Unit/GPU/integration/golden suites pass.
Performance controlled	Baselines and regression thresholds exist.

22. Required Radiance Illumination Debug Panel

Create an editor panel named Radiance Illumination. It is the operational surface for authoring, inspecting, profiling and diagnosing v0.6 while reading production resources without changing ownership.

Overview

- ☐ Enabled state, quality preset and capability tier.
- ☐ Viewport resolution/HDR format.
- ☐ Active/visible/shadowed lights.
- ☐ Environment, exposure and total GPU time.

Surface and materials

- ☐ Base color, normals, roughness, metallic, AO, emissive.
- ☐ Object/material ID, resolved handles and fallbacks.
- ☐ Velocity and transform history.

Direct lighting

- ☐ Light-type contributions, direct diffuse/specular.
- ☐ Reference/clustered difference.
- ☐ Cluster occupancy and overflow.

Shadows

- ☐ Directional/local atlas, cascade and face views.
- ☐ Bias, filter, cache age, dirty reason and denial.
- ☐ Per-view workload/timing.

Environment/reflections

- ☐ Sky, irradiance, prefilter mips, BRDF LUT.
- ☐ Probe selection/blend.
- ☐ SSR hit, confidence, history and source.

AO

- ☐ Raw, filtered and temporal AO.
- ☐ History weight and rejection.

Exposure/color

- ☐ Histogram, current/target exposure.
- ☐ HDR, exposed, tone-mapped and final output.
- ☐ Bloom levels and grading.

Atmosphere/temporal

- ☐ Fog density, transmittance, scattering and slices.
- ☐ Velocity, disocclusion, history weight and reset reason.

Resources/profiling

- ☐ Format, extent, generation, owner and layout.
- ☐ Memory, capacity, high-water marks and overflow.
- ☐ CPU/GPU timings and CSV capture.

23. Required Visualization Modes

Visualization mode	Visualization mode
☐ Base color	☐ World/view/geometric/shading normals
☐ Tangent/bitangent	☐ Roughness
☐ Metallic	☐ Material AO
☐ Emissive	☐ F0
☐ Material ID	☐ Object ID
☐ Linear depth	☐ World position
☐ Velocity	☐ Direct diffuse/specular
☐ Light-type contributions	☐ Diffuse/specular environment
☐ Shadow factor	☐ Cascade index
☐ Receiver bias	☐ Local shadow owner
☐ Cluster index/occupancy	☐ Raw/filtered/temporal AO
☐ Probe index/weight	☐ SSR steps/hit/confidence/source
☐ HDR luminance	☐ Exposure
☐ Bloom levels	☐ Tone-mapped/final output
☐ TAA weight/disocclusion	☐ Fog density/transmittance
☐ Volumetric scattering/shadow/slice	☐ Conventional versus RVG parity
☐ Overflow status	

24. Recommended Source Structure

This is a logical ownership map. Integrate with existing Omnix directories where possible; do not perform destructive reorganization merely to match it.

```

Rendering/Illumination/
|-- Core/          RadiancePipeline, Types, Settings, Capabilities, ResourceContracts
|-- Materials/     PBRMaterialAsset, Registry, GPUBuffer, Fallbacks
|-- Lights/        LightHandle, Registry, Uploader, Components
|-- Shadows/       ShadowSystem, Atlas, Allocator, ViewBuilder, Cascades, Cache
|-- Environment/   SkyLight, EnvironmentAsset, Importer, Irradiance, Prefilter, LUT
|-- Reflections/   ProbeComponent, ProbeRegistry, Capture, Resolve, SSR
|-- Atmosphere/    Settings, HeightFog, FogVolume, FoxelGrid, History
|-- Exposure/      Controller, Histogram, ToneMap, ColorGrading
|-- Temporal/      HistoryManager, Velocity, TAA, Validation
|-- Passes/        Direct, Environment, Composition, AO, Sky, Fog, Volume, Bloom, ToneMap
`-- Debug/         Panel, Visualizer, Inspector, Profiler, Overflow, CaptureTools

```


25. Recommended Implementation Sequence

1. Repair depth, reconstruction, ID and synchronization.
2. Formalize shared surfaces and validation scene.
3. Stabilize materials, tangents, fallbacks and BRDF.
4. Implement reference direct lighting.
5. Harden clustered lighting against reference.
6. Implement directional cascades.
7. Add HDR sky and IBL with manual exposure.
8. Correct AO and composition.
9. Add local-shadow atlas, spot and point shadows.
10. Add reflection probes.
11. Build HZB and SSR with fallback.
12. Add histogram exposure, final tone-map and grading.
13. Write velocity, add bloom and TAA.
14. Add analytic fog then froxel volumetrics.
15. Complete editor tooling, quality tiers and fallbacks.
16. Add reload/recreation, tests and production hardening.

First visual milestone

Use a grey floor, dielectric/metal spheres, one sun, stable cascaded shadow, HDR sky, diffuse/specular IBL, manual exposure and filmic tone mapping. Do not add SSR, bloom or volumetrics to hide defects in this proof scene.

26. Premature Implementations to Avoid

- [] SSR before depth reconstruction and HZB are correct.
- [] Cascades before common view data and depth-only geometry are stable.
- [] Auto exposure before manual exposure/HDR/display transfer.
- [] Temporal AO before current-frame AO and valid velocity.
- [] TAA before current/previous transforms and motion vectors.
- [] Volumetrics before direct light/shadow/HDR composition.
- [] Shadow caching before uncached local shadows work.
- [] Probe blending before one static probe works.
- [] Bit-packing before channel parity/inspection.
- [] RVG-specific lighting or a separate material model.
- [] Full GI, path tracing, neural rendering or mandatory ray tracing.
- [] A second frame loop, scene, depth, material registry or presentation path.

27. Final Completion Criteria

- [] Radiance remains integrated with existing Renderer and RenderGraph.
- [] Conventional and RVG surfaces use one surface and lighting contract.
- [] Depth, position, spaces and color transfer are correct.
- [] Picking uses integer ID.
- [] PBR materials and fallbacks are stable.
- [] Directional/point/spot lighting is correct in linear HDR.
- [] Clustered lighting matches reference and reports overflow.
- [] Directional and local shadows are stable and budgeted.
- [] Sky/IBL light shadows and metals correctly.
- [] AO affects ambient visibility only.
- [] Probes work and SSR falls back without black holes.
- [] Exposure/tonemap/grading/display form one pipeline.
- [] Bloom uses HDR energy.
- [] Velocity and temporal resolve are stable.
- [] Fog/volumetrics respect depth, shadows and history reset.
- [] Every major resource/work list is inspectable and timed.
- [] Quality tiers preserve authored meaning.
- [] Resize, reload and recreation work.
- [] Exhaustion/missing assets degrade safely.
- [] Automated parity, visual and stress tests pass.

Appendix A - Pass and Resource Contracts

Pass	Required reads	Required writes
GBuffer	Instances, materials, textures	Surface targets including depth, ID and velocity.
Directional Shadow	Views, casters, geometry	Directional atlas depth.
Local Shadow	Scheduled views, casters	Local atlas/cubemap depth and cache.
AO	Depth, normals, camera	Raw/filtered/temporal AO.
Light Culling	Depth, lights	Cluster ranges, indices and state.
Direct Lighting	Surfaces, lists, lights, shadows	DirectLightingHDR.
Environment	Surfaces, sky/probes, AO	IndirectLightingHDR.
SSR	Depth/HZB, surface, HDR, probes, history	ReflectionHDR/confidence/history.
HDR Composition	Direct, indirect, reflection, emissive	SceneColorHDR.
Atmosphere	HDR, depth, lights, shadows, fog	Composited HDR/history.
Transparent	HDR, depth, queue, lighting	HDR with transparency.
Exposure	HDR	Histogram/target exposure.
Bloom	HDR	Bloom pyramid/composite.
Tone Map	HDR, exposure, bloom, grade	Display color.
Temporal Resolve	Current, velocity, depth, history	Resolved color/history.
Editor Overlay	Final/depth/ID/selection	Viewport before UI.

Appendix B - Vulkan Synchronization Checklist

- ☐ GBuffer/depth writes visible before shader reads.
- ☐ Shadow depth writes visible before comparison sampling.
- ☐ Cluster compute writes visible before lighting reads.
- ☐ AO writes visible before environment composition.
- ☐ Environment preprocessing visible before cubemap sampling.
- ☐ HDR writes visible before SSR, atmosphere, bloom and exposure.
- ☐ HZB mip writes visible to later reductions and consumers.
- ☐ History writes visible next frame; generations prevent resized reuse.
- ☐ Final color visible before ImGui/presentation.
- ☐ Descriptor slots, atlas tiles and resources reused only after referencing frames complete.
- ☐ Queue ownership transfers explicit.
- ☐ Layouts do not replace memory dependencies.
- ☐ ALL_COMMANDS barriers require justification.
- ☐ Synchronization validation enabled for exit tests.

Appendix C - Agent Milestone Report Template

MILESTONE: L_ - _____
DATE / COMMIT:
AGENT:

1. Scope completed
2. Modified / new / deleted files
3. Architecture changes
4. Resource / descriptor / RenderGraph changes
5. Build and runtime result
6. Vulkan validation result
7. Debug views and editor controls
8. Tests and golden/parity evidence
9. GPU / CPU performance
10. Memory / capacity measurements
11. Failure-injection results
12. Known limitations and blockers
13. Recommended next task

EXIT GATES

☐ All milestone gates passed.
☐ Renderer remains buildable and scenes load.
☐ No unreported overflow or validation error remains.
☐ Required documentation updated.

Final engineering standard

Radiance v0.6 is complete only when the image is produced by repeatable, documented and inspectable systems that remain safe under editor use, reduced capability, capacity pressure, resize, reload and failure. Visual beauty is an output of correct contracts, not a substitute for them.